



Program: BS Computer Science

Semester: Fall 2025

Credit Hour(s): 1

Pre-requisite(s): EE1005 Digital Logic Design

Instructor: Engr. Muhammad Hassaan Bin Shoukat – hassaan.shoukat@nu.edu.pk

Lab # 9: Masking Operations & Subroutines

1. Objectives

- Understand and implement **masking operations** for selective bit manipulation.
- Write and call **subroutines** using CALL and RET.
- Understand how stack support subroutine execution.

2. Masking Operations

Masking is a technique used to *manipulate specific bits* of a data byte or word. It involves performing *logical operations* between a **data value** and a **mask**.

A mask is a specially designed binary pattern that determines which bits are affected and which remain unchanged. Masking can be used to **clear**, **set**, **invert** or **test** specific bits.

2.1. Selective Bit Clearing (Using AND)

The AND operation can be used to **make specific bits zero** (clear them) while keeping others unchanged.

A mask is created with 1's in positions that should *retain* their original value and 0's where bits should be *cleared*.

Example 1: To clear the lower nibble of AL, apply:

```
[org 0x0100]
```

```
jmp start
```

```
masked_val: db 0
```

```
mask_lower:
```

```
    and al, 0F0h                ; Mask lower nibble (keep upper nibble)
```

```
    mov [masked_val], al        ; Store result
```

```
    ret
```

```
start:
```

```
    mov al, 9Ah
```

```
    call mask_lower
```

```
mov ah, 4Ch
```

```
int 21h
```



- The **upper nibble** remains unchanged
- The **lower nibble** becomes zero

2.2. Selective Bit Setting (Using OR)

The OR operation can be used to *set specific bits* (make them 1) while keeping others unchanged.

The mask has 1's at positions that need to be *set* and 0's where bits should be *preserved*.

Example: To set the lower nibble of AL:

```
or al, 0Fh
```

- The **upper nibble** retains its value
- The **lower nibble** becomes all 1's

2.3. Selective Bit Inversion (Using XOR)

The XOR operation can be used to *invert specific bits* of a number. Bits corresponding to 1's in the mask are *toggled*, while bits corresponding to 0's remain *unchanged*.

Example: To invert the lower nibble of AL:

```
xor al, 0Fh
```

- The **upper nibble** remains unchanged
- The **lower nibble** is inverted

This is different from NOT, which inverts all bits in a register. XOR allows *selective inversion*.

2.4. Selective Bit Testing (Using AND or TEST)

The AND instruction can be used to *check whether certain bits of a number are set or not*. However, it destroys the original destination value.

To avoid this, the TEST instruction is used, it performs a *non-destructive AND* (it only affects flags)

- If the result is zero, the **Zero Flag (ZF)** is set.
- If the result is non-zero, ZF is cleared.

Syntax: result = destination AND source

Example:

```
mov bx, 1
test bx, [multiplier]
jz skip
```

This checks whether the least significant bit of [multiplier] is 1. If not, the program jumps to label skip.

Example 2: Selective Bit Testing in Multiplication

```
[org 0x0100]
jmp start
```

```
multiplicand: dd    1300    ; 16-bit multiplicand (32-bit space)
multiplier:   dw     500      ; 16-bit multiplier
result:       dd     0        ; 32-bit result
```



start:

```
mov cl, 16      ; initialize bit count to 16
mov bx, 1       ; initialize bit mask
```

checkbit:

```
test bx, [multiplier] ; test current bit of
multiplier
jz skip           ; skip addition if bit is zero

mov ax, [multiplicand]
add [result], ax  ; add lower word
mov ax, [multiplicand+2]
adc [result+2], ax ; add upper word with carry
```

skip:

```
shl word [multiplicand], 1
rcl word [multiplicand+2], 1
    ; shift multiplicand left
shl bx, 1           ; move mask to next bit
dec cl              ; decrement bit count
jnz checkbit        ; repeat if bits left

mov ax, 0x4c00
int 0x21
```

Benefit of using this approach:

- The TEST instruction checks each bit of the multiplier without changing its value.
- The corresponding bit in the mask (BX) determines whether addition is performed.
- This technique replaces the need for repeated shifting and carry checking.

3. Subroutines

A subroutine (or procedure in assembly) is a self-contained block of code that performs a specific task. It is a fundamental concept in programming that promotes code modularity and reusability.

Instruction	Meaning	Description
CALL procedure_name	Call	Transfers control to the procedure. It pushes the address of the instruction <i>following</i> the CALL onto the Stack (the Return Address), then jumps to the procedure's starting address.
RET	Return	Transfers control back to the instruction immediately following the most recent CALL. It <i>pops</i> the Return Address off the Stack and jumps to that address.



Program Flow:

Main Program → CALL Subroutine → RET → Back to Main Program

3.1. The Stack

The **Stack** is an area of memory used for temporary data storage and, most importantly, for managing subroutine calls. It is a *Last-In, First-Out (LIFO)* data structure.

Key Stack Registers

- **Stack Segment (SS):** Holds the segment address of the Stack.
- **Stack Pointer (SP):** Holds the offset of the topmost element (the last item *pushed*) in the Stack. It points to the currently occupied word.

Key Instructions

Instruction	Description
PUSH source	<i>Decrements the SP by 2, then copies the 16-bit value from the source onto the Stack at the new SP location.</i>
POP destination	<i>Copies the 16-bit value from the Stack location pointed to by SP into the destination, then Increments the SP by 2</i>

Important: The Stack grows **downwards** (from higher addresses to lower addresses) in memory.

3.2. Preserving Registers

When a subroutine is called, it often needs to use registers like **AX, BX, CX, DX**. If the main program needs the *original* values of those registers after the subroutine returns, the subroutine must:

- **PUSH** the registers it will use at the beginning.
- **POP** the registers it pushed (in **reverse order**) just before the RET instruction.

Example 3: Observe how the stack operates with PUSH and POP.

```
[org 0x0100]
    mov ax, 1234h
    mov bx, 5678h

    push ax
    push bx

    mov ax, 0
    mov bx, 0

    pop bx
    pop ax
    mov ah, 4Ch
    int 21h
```



Example 4: Bubble Sort Using Subroutine

```
[org 0x0100]
        jmp start

data:    dw    60, 55, 45, 50, 40, 35, 25, 30, 10, 0
swap:    db    0

bubblesort:
        dec    cx                ; last element not compared
        shl    cx, 1             ; turn into byte count

mainloop:
        mov    si, 0             ; initialize array index to zero
        mov    byte [swap], 0    ; reset swap flag to no swaps

innerloop:
        mov    ax, [bx+si]        ; load number in ax
        cmp    ax, [bx+si+2]      ; compare with next number
        jbe    noswap            ; no swap if already in order

        mov    dx, [bx+si+2]      ; load second element in dx
        mov    [bx+si], dx        ; store first number in second
        mov    [bx+si+2], ax      ; store second number in first
        mov    byte [swap], 1    ; flag that a swap has been done

noswap:
        add    si, 2             ; advance si to next index
        cmp    si, cx            ; are we at last index
        jne    innerloop         ; if not compare next two

        cmp    byte [swap], 1    ; check if a swap has been done
        je     mainloop          ; if yes make another pass

        ret                    ; return to caller

start:
        mov    bx, data          ; send start of array in bx
        mov    cx, 10            ; send count of elements in cx
        call   bubblesort        ; call our subroutine
mov    ax, 0x4c00                ; terminate program
int    0x21
```

Stack and Subroutine Relationship

When CALL bubblesort executes:

1. CPU pushes the *next instruction address* on the stack.
2. CPU jumps to the *bubblesort procedure*.



3. When RET executes, the *address is popped* from the stack, and execution resumes in the main program.

This demonstrates *stack-based control flow management*.

4. Lab Tasks

Task 1: Selective Bit Operations

Write a program that performs the following operations on AL = 9Ah:

1. Clear the lower nibble using AND.
2. Set the lower nibble using OR.
3. Invert the lower nibble using XOR.

(Each operation should be written as a separate code block for clarity.)

Task 2: Selective Bit Testing using TEST

Write a program that:

1. Stores a 16-bit number in AX.
2. Uses the TEST instruction to check whether **bit 3** is set or not.

(Hint: Use TEST AX, 00001000b and conditional jumps like JZ or JNZ.)

Task 3: Subroutine for Selective Masking

Write a program that:

1. Takes a byte in BL.
2. Calls a subroutine *clear_lower* that clears the lower nibble.
3. Returns the masked value in BL.

Task 4: Subroutine for Sum Calculation

Write a program that:

1. Defines an array of 10 numbers.
2. Calls a subroutine named *sum_array* that calculates the total sum.
3. Stores the sum in memory.
4. Returns control to the main program after completion.

Task 5: Stack-Based Value Exchange

Write a program that:

1. Initializes AX = 1111h and BX = 2222h.
2. Pushes both registers onto the stack.
3. Pops them in reverse order (to exchange their values).
4. Verify that AX and BX have swapped their contents.